makemore

Build a bigram LM. Looking at 1 character and try to predict next character. Forget about charaters further away more than 1.

GOAL: read file and put words into list.

6:27 open file in 1 line.

```
words =  open("names.txt", "r").read().readline()
```

GOAL: split words in list into bigrams

10:38 b add stop and end tokens to sentence, convert the word to list of chracters, zip through the args to create 2 iterators. There are 2 iterators in zip and they move in lock step with each other to produce the values ch1 and ch2

```
for w in words:
  chs = ['<S>']+list(w)+['<E>']
  for ch1, ch2 in zip(chs, chs[1:]:
     print(ch1,ch2)
```

GOAL: create a dict of bigrams to remove the duplicate bigrams

`b={}`

This is probably the most common API pattern in python dicts, if key not there add it to the dictionary with a value of 1 and if the key is there increment the count of the value. the verbose way to do this is:

```
d={}
if "a" not in d:
  d["a"]=1
 else:
  d["a"]=d["a"]+1
```

A couple things:

what is better? `d["a"]` or `d.get("a")`? the first is less typing but has an exception if key not found. The second one is better because it provides a default value if the key isn't found. `d.get("a", default_value).`

Use `setdefault` instead of get if want to write into the dictionary. If key not there we want to set the key to the default value.

```
d.setdefault("my_key", "new_value")
```

GOAL: create a dict of bigrams

```
b={}
for w in words:
  chs = ['<S>'] + list(w) + ['<E>']
   # one iter on chs from index 0 another iter on substring chs
from index 1
  for ch1, ch2 in zip(chs, chs[1:]):
    bigram = (ch1, ch2)
    b[bigram] = b.get(bigram,0) + 1
```

GOAL: convert dict to list to print top 10. Can't print out first 10 entries in dict.

```
b={"a":10, "b":11}
list(b.items())[:10]
```

Convert using `items()`. Leaving `items()` off will only add keys to the list. Using list(b) is not correct because the default leaves off the values and only adds the keys.

GOAL: sort the list by values. First sort by keys. `sorted(b.items())` Then adjust the keys to be values. `sorted(b.items(),key=lambda x:x[1])` then adjust to list in descending order `sorted(b.items(), key=lambda x:-x[1])`

```
sorted(b.items(), key=lambda x:-x[1])
```

`13:45` GOAL: convert sorted list to 2d array
Create a 2d torch array and specify dtype torch.int32 for counts in dict. Default the dtype is set to torch.float32.

```
a = torch.zeros(28,28, dtype=torch.int32)
```

We are going to use each column/row to represent a letter. There are 26 letters + EOS and BOS. Each bigram is a cell in the 2d array.

GOAL: convert char to int for indexing into pytorch 28,28 array.
STEPS: create a single long string from all the words, then tokenize and convert to set. This removes the spaces and duplicates.

```
ch = set(''.join(words))
```

Error check there are 26. Then add the start and end tokens

Enumerate to show index with char

```
for idx, ch in enumerate(ch):
  print(idx,ch)
```

```
<class 'list'>
0 a
1 b
2 c
3 d
4 e
5 f
6 g
7 h
8 i
9 j
```

```
10 k
11 l
12 m
13 n
14 o
15 p
16 q
17 r
18 s
19 t
20 u
21 v
22 w
23 x
24 y
25 z
```

create a dictionary to use as lookup
17:25 GOAL: convert to dictionary comprehension

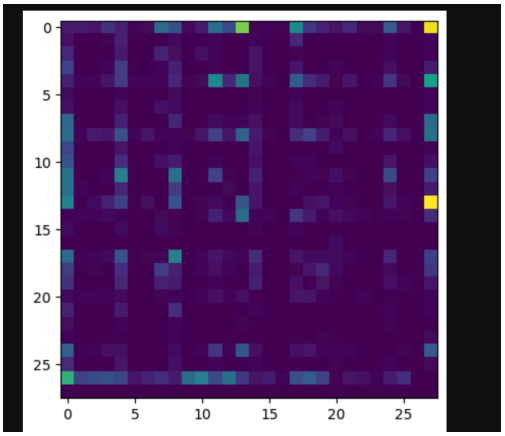
```
stoi = {ch:idx for idx, ch in enumerate(chars) }
```

GOAL: add the start and stop tokens

```
stoi['[<S>]'] = 26
stoi['[<S>]'] = 27
```

GOAL; create teh 2d torch matrix and add counts of each bigrm into the matrix.

Try to visualize better. WIthout doing anything but plotting N we get
`plt.imshow(N)`



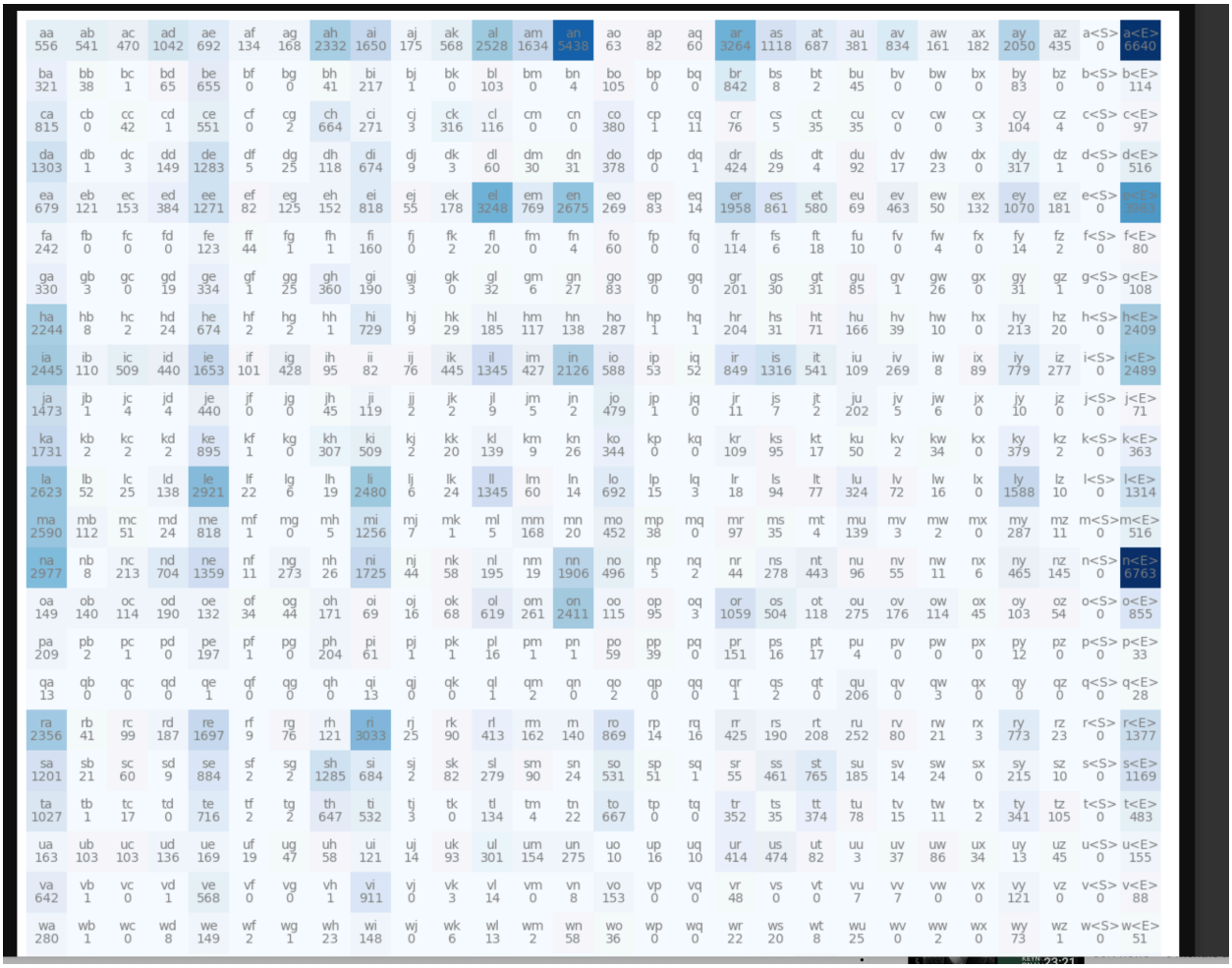
18:41: We need a reverse lookup from letter to int. Create a reverse lookup following the same process. We want to list the chars, index and form a mapping from index to char.

```
for _, ch in enumerate(stoi):
  print(ch, stoi[ch])
```

Use a dictionary comprenehsion and iterate through the dict using dict.items(). k.v is same as x[0], x[1]

```
itos = {x[1]:x{0} for x in stoi.items()}
```

20:55 karpathy has following matplotlib



21:14 there are many zeros at the end or bottom of the plot. Because the end token wont be next to anything. And teh S column will never be the second character in a bigram. So there are some ineffiencies.

22:14 GOAL: convert 2 EOS and BOS to 1 token. Substitute <S> and <E> with . Replace with dot. That isn't. period for end of sentence.

27:11 convert counts to probabilities of first row on N

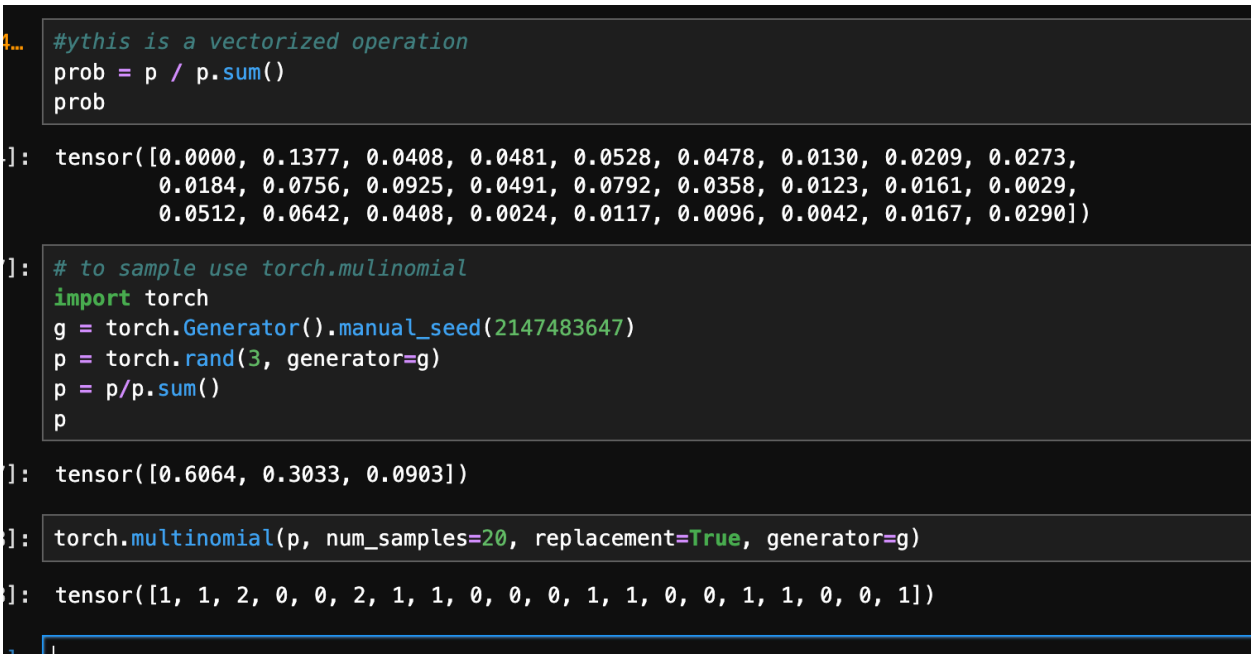
28:50 create a probability distribution for 3 categories. These 3 categories are random 3 categories.

```
p = torch.Generator().manual_seed(42)
p = torch.rand(3, generator=g)
p = p.sum()
```

Can see how all probs add up to 1.

To sample from these 3 categories randomly

torch.sample(p, num_samples=20, replacement=True, generator=g)



```
#ythis is a vectorized operation
prob = p / p.sum()
prob
```

```
tensor([0.0000, 0.1377, 0.0408, 0.0481, 0.0528, 0.0478, 0.0130, 0.0209, 0.0273,
        0.0184, 0.0756, 0.0925, 0.0491, 0.0792, 0.0358, 0.0123, 0.0161, 0.0029,
        0.0512, 0.0642, 0.0408, 0.0024, 0.0117, 0.0096, 0.0042, 0.0167, 0.0290])
```

```
# to sample use torch.mulinomial
import torch
g = torch.Generator().manual_seed(2147483647)
p = torch.rand(3, generator=g)
p = p/p.sum()
p
```

```
tensor([0.6064, 0.3033, 0.0903])
```

```
torch.multinomial(p, num_samples=20, replacement=True, generator=g)
```

```
tensor([1, 1, 2, 0, 0, 2, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1])
```

As a correctness check we would expect 60% of the samples to be 0s. 30% one, 9% 2s.

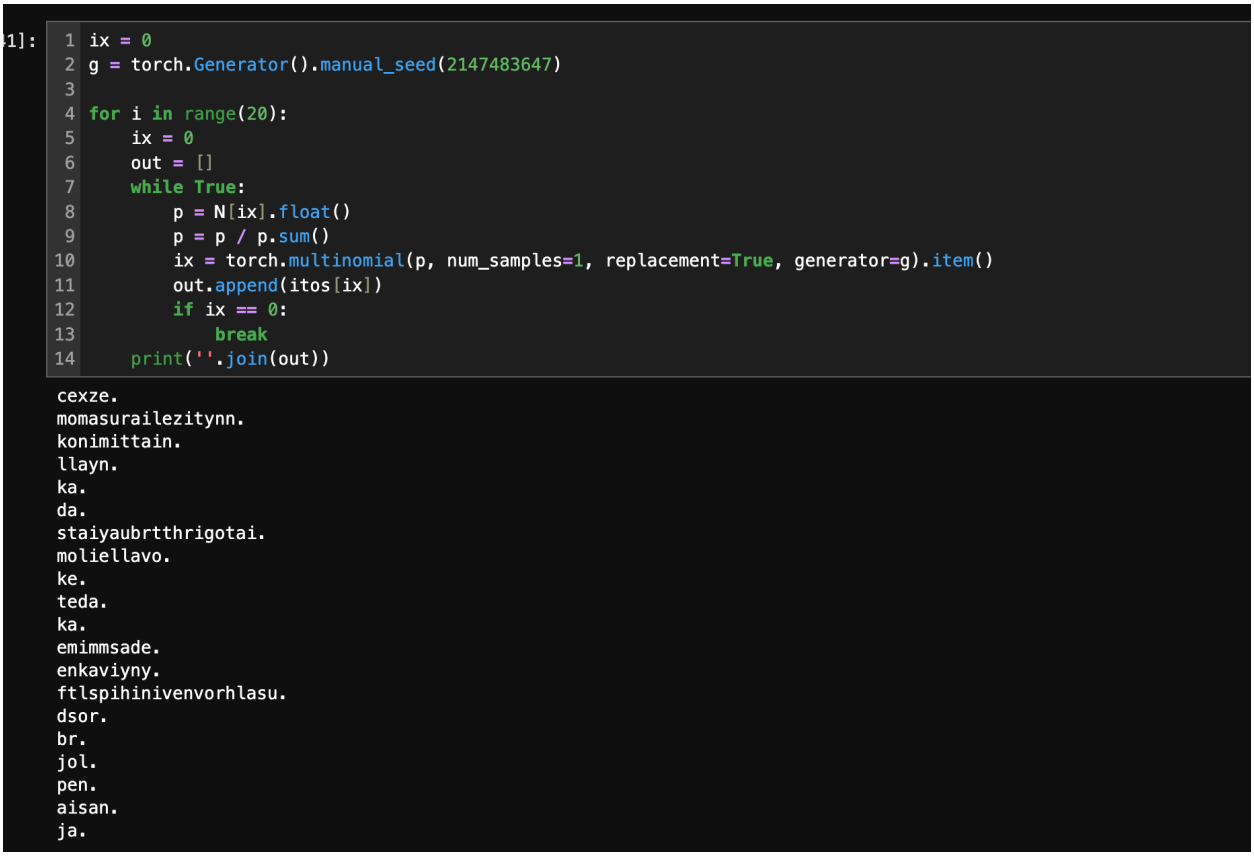
30:17 GOAL: we want to sample from our p we calculated from N. This means calling torch.multinomial(p..) use the p from p = N[0].float(); p = p/p.sum(). p is vectorized.

```
g = torch.Generator().manual_seed(...)
ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g)
```

We see ix is a type tensor. Use .item() to pop out the integer

```
ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
```

31:51: GOAL: write out a loop to predict bigrams. Sample a char from row then pick out second char after first char sampled.



```
1]:
ix = 0
g = torch.Generator().manual_seed(2147483647)

for i in range(20):
    ix = 0
    out = []
    while True:
        p = N[ix].float()
        p = p / p.sum()
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

```
cexze.
momasurailezitynn.
konimittain.
llayn.
ka.
da.
staiyaubrtthrigotai.
moliellavo.
ke.
teda.
ka.
emimmsade.
enkaviyny.
ftlspihinivenvorhlasu.
dsor.
br.
jol.
pen.
aisan.
ja.
```

Can't match the YT strings. Probably where couldn't match "m"

Goal: 36:17 compare uniform distribution where there is no bigram or training.

```
2]:
ix = 0
g = torch.Generator().manual_seed(2147483647)

for i in range(20):
    ix = 0
    out = []
    while True:
        # p = N[ix].float()
        # p = p / p.sum()
        p = torch.ones(27)/27.
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

```
cexzm.
zoglkurkicqzktyhwmvmzimjttainrlkfukzkktda.
sfcxvpubjtbhrmgotzx.
iczixqctvujkwptedogkkjemkmmsidguenkbvgynywftbspmhwcivgbvtahlvsu.
dsdxxblnwglhpyiw.
igwnjwrpfdwipkwzkm.
desu.
firmt.
gbiksjbquabsvoth.
kuysxqevhcmrbxmcwyhrrjenvxmvpfkmwmghfvjzxobomysox.
gbptjapxweegpfwhccfyzfvksiiqmvwbhmiwqmdgzqsamjhgamcxwmmk.
iswcxfmbalcslhy.
fpycvasvz.
bqzazeunschck.
wnkojuoxyvtvfiwksddugnkul.
fuwfcgjz.
abl.
j.
nuuutstofgqzubbo.
rdubpknhmd.
```

Can't really tell difference except bigram length is shorter indicating closer to name generation.

Goal: 36.27 fix inefficiency in N[ix] where we are stepping through each row of N, converting to float then sum and divide. add broadcasting where we create a prob matrix P.

Goal: 40.28 P.sum(0,keepdim=True) vs. the default P.sum(0,keepdim=False). If true it squeezes dim out. but how? Some examples

X=[1,2][3,4]
A X.sum(dim=0) is a columm, [,]

A X.sum(dim=1) is row. []

43:43 broadcasting copies the 1 dimension to a longer array to match the dimensions of the other array.
Broadcasting rules, dimensions have to be same, non existent, or 1. What does this mean? Line up the dimensions by column like doing math

```
27 27
27 1
```

The 1 dimension is copied to match 27.

Check your work. Verify the correct dimension is being broadcast.

If there is no dimension,

```
27 27
   27
```

then will insert a 1 where it is blank

Would expect this to work

```
P = N.float()
P = P/P.sum(1)
```

This is a bug. doesnt work

```
P = N.float()
P = P/P.sum(1, keepdim = True)
```